

TraceChain

Cryptocurrency fraud tracing, from a victim-reported wallet to the exchange it was cashed out through
Smart India Hackathon 2026 · Problem Statement **SIH26183** · Ministry of Home Affairs

A victim reports one wallet address. TraceChain follows the money outward hop by hop, flags known laundering patterns, identifies the exchange where the funds landed, and produces a report an investigator can attach to a legal request. **Every conclusion is re-checkable by hand** — attribution is an exact match against a published wallet list, and every laundering finding is arithmetic that reports the thresholds it applied. No model, no clustering, no proprietary score, because a conclusion that reaches a courtroom must be one a human can verify.

337 VERIFIED EXCHANGE WALLETS	18 EXCHANGES COVERED	4 HOPS TRACED BY DEFAULT	3 WEIGHTED CONFIDENCE FACTORS
----------------------------------	-------------------------	-----------------------------	----------------------------------

HOW IT WORKS

1 · Fetch Rate-limit-aware Etherscan V2 client; retries with exponential backoff	2 · Traverse BFS over outgoing transfers, depth-limited, highest-value first, dust dropped	3 · Attribute Exact match against the labelled exchange wallet database	4 · Detect Peel chain & amount split, as fixed arithmetic rules	5 · Score Hop proximity 40% + amount correlation 35% + match directness 25%	6 · Report Case stored in SQLite; exportable as JSON or plain-text report
--	--	---	---	---	---

CURRENT STATUS — VERIFIED WORKING

CAPABILITY	STATE	DETAIL
Ethereum native tracing	Working	Live test: reported address → Binance 14 in 2 hops, confidence 0.80 (high) ; 138 addresses mapped at 4 hops
Exchange attribution	Working	337 wallets / 18 exchanges, each proven on-chain to carry real value
Laundering detection	Working	Peel chain and amount split; each finding reports its own thresholds
Confidence scoring	Working	Three weighted components, each with a written explanation in the report
Interactive graph	Working	Bubble map: size = value moved, drag-to-pin, hover isolates counterparties, zoom/fit
Report export	Working	JSON + plain text, incl. methodology and an explicit limitations section
Multi-chain architecture	Built	Chain registry + provider seam; BSC registered, awaiting a data source
BSC / other chains (data)	Blocked	Etherscan free tier serves Ethereum mainnet only — verified against the live API
ERC-20 / stablecoin tracing	Not built	Highest-value gap: most laundering moves as USDT/USDC, not native ETH

DESIGN GUARANTEE — WHY IT HOLDS UP IN COURT

Attribution names **where funds arrived, not who controls the account**. Only the exchange can link a deposit address to a customer identity, via lawful request — and the generated report states this explicitly rather than leaving it implied. Every attribution traces to a specific, checkable line in a data file; every threshold is printed alongside the finding it produced. This is a deliberate constraint, not a missing feature.

TECHNOLOGY & REPOSITORY

BACKEND — PYTHON / FASTAPI

- `etherscan_client.py` — throttled API client, backoff, three-way handling of "empty vs rate-limited vs error"
- `graph_builder.py` — BFS with four documented brakes
- `exchange_matcher.py` — per-chain exact-match attribution
- `pattern_detection.py` · `scoring.py` · `report.py`
- `chain_data.py` — provider seam for additional chains

FRONTEND — REACT 19 / VITE

- `GraphView.jsx` — force-directed canvas, value-proportional bubbles, collision packing, drag-to-pin
- `ResultSidebar.jsx` — attribution, confidence breakdown, trace path, limitations, export
- Production bundle: 400 KB (128 KB gzipped)

REPRODUCIBLE DATA TOOLING

- `import_exchange_labels.py` — imports and verifies labels
- `audit_label_contracts.py` — strips non-custody contracts
- `verify_labels.py` — re-checks the database on demand

WHAT THE PROJECT NEEDS TO FUNCTION

RUNTIME PREREQUISITES

- **Python 3.11+** — FastAPI, NetworkX, SQLAlchemy, httpx
- **Node.js 20+** — React 19, Vite, react-force-graph-2d
- **Etherscan API key** (free tier) in `backend/.env`
- Outbound HTTPS to `api.etherscan.io`

RUNNING IT

- Backend: `uvicorn app.main:app --reload --port 8000`
- Frontend: `npm run dev` → `http://localhost:5173`
- Use `localhost`, not `127.0.0.1` — Vite binds IPv6
- Backend must be up first; the UI proxies `/api/*` to `:8000`

OPERATIONAL LIMITS (BY DESIGN)

- **Depth** 4 hops default, 6 max — laundering that matters happens close to the source
- **Fan-out** 10 destinations per address, highest value first
- **Dust floor** 0.001 native units — spam is not followed
- **Node cap** 400 addresses per trace
- **Terminal stop** at known exchange wallets — the trace has its answer

DATA INTEGRITY CONTROLS

- Every label verified on-chain before acceptance (37 rejected)
- Contract audit removed 4 token/DEX-router addresses wrongly labelled as exchanges
- Both import scripts are re-runnable and pinned to a dataset commit

KNOWN GAPS TO CLOSE BEFORE PRODUCTION

GAP	SEVERITY	CONSEQUENCE AND FIX
No ERC-20 / token tracing	CRITICAL	A wallet that moved stolen USDT reports "nothing to trace" — a false negative on the most common laundering pattern. Etherscan's <code>tokenTx</code> action is free and already available on the current key. Requires per-asset value handling in scoring.
Single chain only	CRITICAL	Funds bridged or cashed out on BSC/Tron/Polygon are invisible. Architecture is ready; only a data source is missing.
API key leaks into logs	HIGH	The HTTP client logs full request URLs including <code>apikey=</code> . Must be suppressed before logs are shared or the repo is pushed.
Rate limit mismatch	HIGH	Client throttles to 4 req/s but Etherscan's free tier now allows 3 req/s ; retry storms were observed under load. One-line fix to the throttle interval.
Label coverage	MEDIUM	Attribution is capped by the wallet list. An exchange not in it returns "no match", which reads like "never cashed out". Indian exchanges (WazirX, CoinDCX) are absent — Etherscan does not label them.
Not yet version-controlled	MEDIUM	No git repository; no history, no rollback, no collaboration. <code>.gitignore</code> is already correct.
SQLite single-file store	LOW	Fine for demo; one writer only. SQLAlchemy makes Postgres a connection-string change.

Operational reality check. A 4-hop trace on an active wallet took **~3 minutes and 59 API calls** under free-tier throttling. For a live demonstration, 2–3 hops is the reliable setting. Throughput — not correctness — is the first thing that breaks at scale, and it is the cheapest thing to fix.

WHERE FUNDING IS REQUIRED TO SCALE

The current build is deliberately zero-cost and therefore capped at one chain, native currency only, and roughly three requests per second. The table below is ordered by **investigative value per rupee**, not by price.

INVESTMENT	COST (VERIFIED 04-09-2026)	PRIORITY	WHAT IT UNLOCKS
Etherscan API — Lite	\$49 / month	CRITICAL	Best value in this table. Unlocks all 60+ chains — BSC, Polygon, Arbitrum, Base — through the key and code already in place. Multi-chain becomes a config change, not an integration. Also raises the limit to 5 calls/sec.
Etherscan API — Standard	\$199 / month	HIGH	10 calls/sec and 200,000 calls/day. Turns a 3-minute trace into well under a minute and supports several concurrent investigators. Upgrade when demo becomes deployment.
Tron data access	Free tier → quote	CRITICAL	USDT-TRC20 is the dominant cash-out rail for Indian scam proceeds. TronGrid has a free tier; sustained investigative volume needs a paid quota. Requires a Tron adapter (different address format and value units).
Commercial attribution data Chainalysis / TRM / Elliptic	Enterprise quote	HIGH	The single biggest capability jump. Adds deposit-address clustering — linking a one-time deposit address to the exchange that owns it — plus sanctions and scam-report data. This is what turns "matched a published wallet" into attribution investigators rely on daily. Budget as a post-pilot line item.
Price / FX data CoinGecko or equivalent	Quote required	MEDIUM	Historical price at each transaction timestamp, so the report states value in INR at the time of theft rather than "3.1 ETH". For a Ministry of Home Affairs filing, the rupee figure is the legally meaningful one.
Hosting & infrastructure	~\$50–200 / month	MEDIUM	Managed Postgres (replacing SQLite), a background job queue so long traces do not block the request, Redis caching to cut repeat API spend, TLS, and backups.
Security & data-protection review	One-off engagement	HIGH	The system handles victim-reported data and produces evidence. Needs a review against India's DPDP Act, an access-control model, and an audit trail of who ran which trace — a prerequisite for any real police deployment, not an optional extra.
Self-hosted archive node	~2 TB NVMe + host	OPTIONAL	Removes per-call API dependency and cost ceilings entirely. Only worth it at sustained high volume; API subscriptions are cheaper below that threshold.

RECOMMENDED PHASING

PHASE 1 — ₹0, THIS WEEK	PHASE 2 — ~\$49/MO	PHASE 3 — DEPLOYMENT
- ERC-20 tracing on Ethereum via <code>tokenx</code> — free on the existing key, and closes the largest correctness gap - Fix the throttle interval and log redaction - Initialise git; commit the verified label database	- Etherscan Lite → BSC, Polygon, Arbitrum live with existing code - Per-asset scoring for USDT/USDC across chains - Tron adapter for TRC-20	- Postgres, job queue, hosting - Security & DPDP review, access control, audit log - Commercial attribution feed for deposit-address clustering

Verification note. Prices marked verified were checked against provider pricing pages on 4 September 2026 and change without notice; items marked "quote required" were not confirmed and must be priced before budgeting. Capability claims marked "Working" were exercised end-to-end against live Ethereum mainnet data during testing. Attribution identifies an exchange, never a person.